Day 4: Jenkins Pipelines & AI-Assisted CI/CD

From freestyle jobs to fully automated Docker-integrated pipelines — powered by AI debugging.

CI/CD TRAINING · DAY 4

Today's Agenda

A full-day journey from Jenkins fundamentals to AI-powered pipeline automation.

Jenkins Jobs

Freestyle & Pipeline job types, execution flow

GitHub Integration

Webhooks, push triggers, SCM setup

Pipeline as Code

Declarative pipelines, Jenkinsfile structure

Docker in Pipeline

Build, tag, and run containers from Jenkins

AI for CI/CD

ChatGPT, Copilot, and AI-assisted debugging

JENKINS JOBS · CHAPTER 1

What Is a Jenkins Job?

A **Jenkins job** is a configurable, repeatable unit of work that Jenkins executes on demand or automatically.

📥 Input

Source code, parameters, or triggers

⚙️ Process

Build, test, lint, package

📤 Output

Artifacts, reports, deployments

JENKINS JOBS · CHAPTER 1

Types of Jenkins Jobs

🛠️ Freestyle Job

GUI-configured. Good for simple builds, shell scripts, and quick experiments. No version control for config.

📄 Pipeline Job

Code-defined via Jenkinsfile. Supports complex multi-stage workflows. Version-controlled alongside your app.

🔁 Multibranch

Automatically discovers branches and PRs. Runs the right Jenkinsfile per branch.

ⓘ For production CI/CD, always prefer **Pipeline jobs** — they're reproducible, reviewable, and scalable.

JENKINS JOBS · CHAPTER 1

Real-World Analogy: The Factory Floor

1

Raw Materials

Your source code arrives via Git

2

Assembly Line

Jenkins stages process and build your app

3

Quality Check

Tests validate correctness before shipping

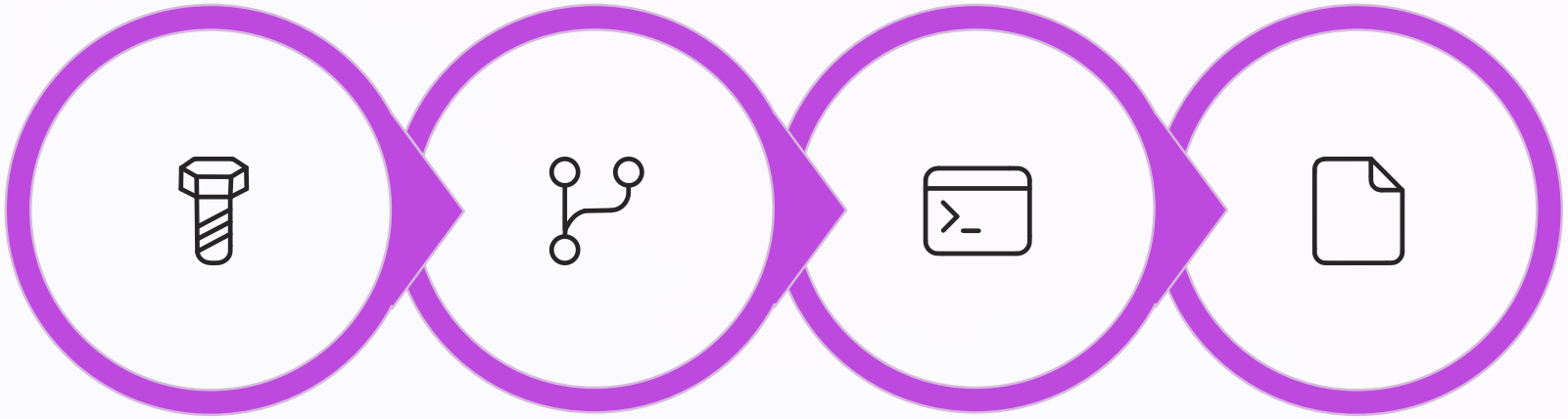
4

Delivery

Artifact or Docker image ships to production

JENKINS JOBS · CHAPTER 1

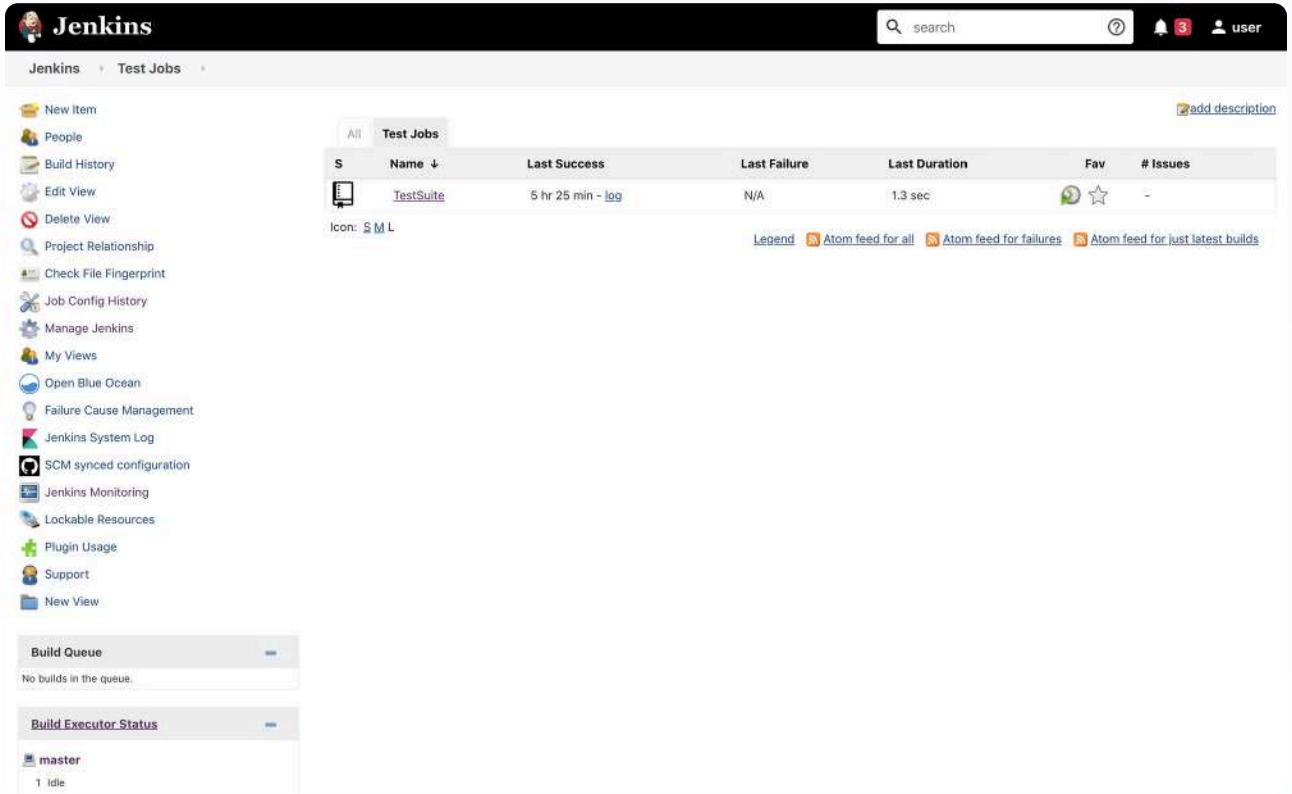
Job Execution Flow



Every Jenkins job — regardless of type — follows this fundamental execution cycle. Understanding it helps you debug failures faster.

FREESTYLE JOBS · CHAPTER 2

Creating Your First Freestyle Job



Step-by-Step

1. Open Jenkins → click **New Item**
2. Enter a job name (e.g., `my-first-job`)
3. Select **Freestyle project** → click OK
4. Configure source, build steps, and save
5. Click **Build Now** to trigger manually

Keep job names lowercase and hyphenated — they appear in URLs and log paths.

FREESTYLE JOBS · CHAPTER 2

Source Code Management: GitHub

Connect your freestyle job to a GitHub repository so Jenkins always builds the latest code.

Repository URL

Paste your GitHub repo URL under **Source Code Management → Git**. Use HTTPS for simplicity in training.

Branch Specifier

Set to `*/main` (or `*/master`) to track your default branch on every build.

Credentials

Add a GitHub Personal Access Token as Jenkins credentials. Never hardcode tokens in the config.

FREESTYLE JOBS · CHAPTER 2

Build Steps: Shell Commands

Under **Build Steps**, add an **Execute shell** step. This runs on the Jenkins agent after checkout.

```
#!/bin/bash
echo "=== Starting build ==="
cd my-app

# Install dependencies
npm install

# Run tests
npm test

# Package the app
npm run build

echo "=== Build complete ==="
```

Exit code `0` = success. Any non-zero exit code marks the build as **FAILED** in Jenkins.

FREESTYLE JOBS · CHAPTER 2

Reading Build Output & Logs

🟢 SUCCESS

All steps completed with exit code 0

🔴 FAILURE

A step failed — check the highlighted error line

🟡 UNSTABLE

Tests ran but some assertions failed

Where to Look

- Click the build number (e.g., `#3`) in the sidebar
- Select **Console Output** for full logs
- Look for lines starting with `+` (commands) and error messages
- Use **Ctrl+F** to search for `ERROR` or `FAILED`

BUILD TRIGGERS · CHAPTER 3

Manual vs. Automatic Builds

🖱️ Manual Trigger

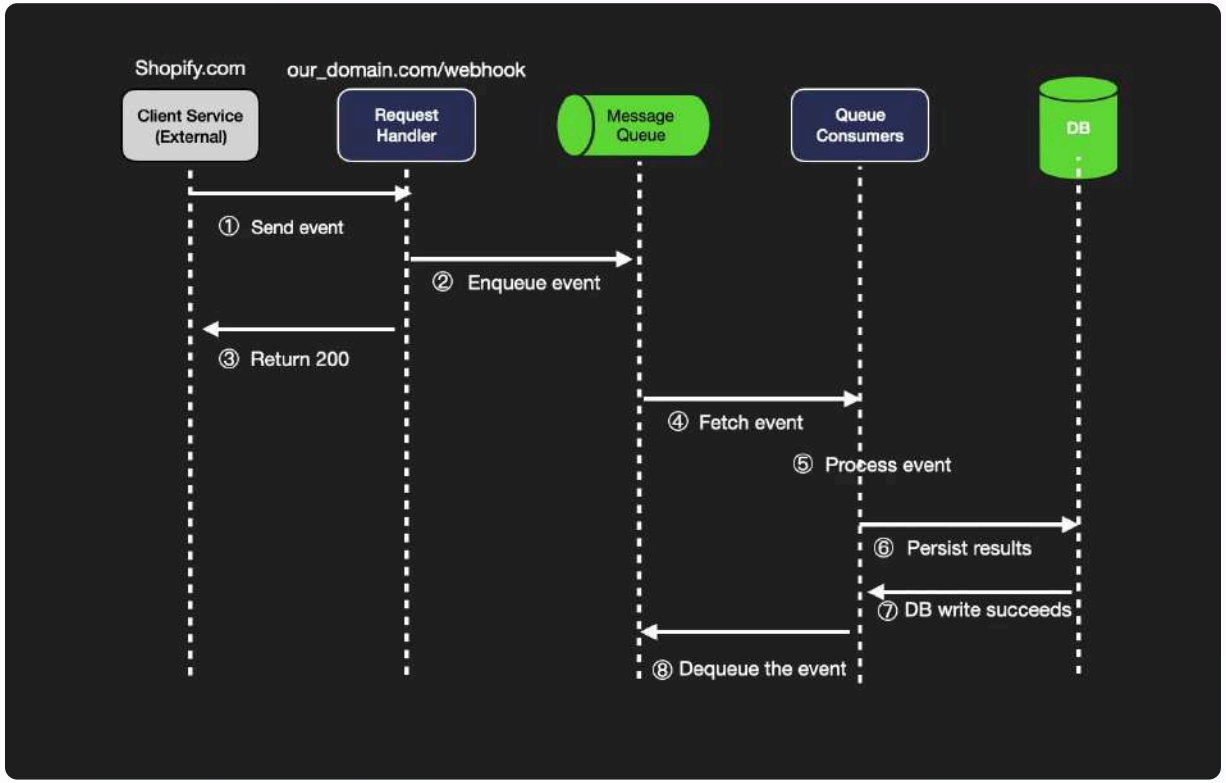
Click **Build Now** in the Jenkins UI. Useful for testing and debugging but doesn't scale for teams.

🤖 Automatic Trigger

Jenkins reacts to events: a Git push, a scheduled cron, or an upstream job finishing. This is the DevOps way.

BUILD TRIGGERS · CHAPTER 3

What Is a Webhook?



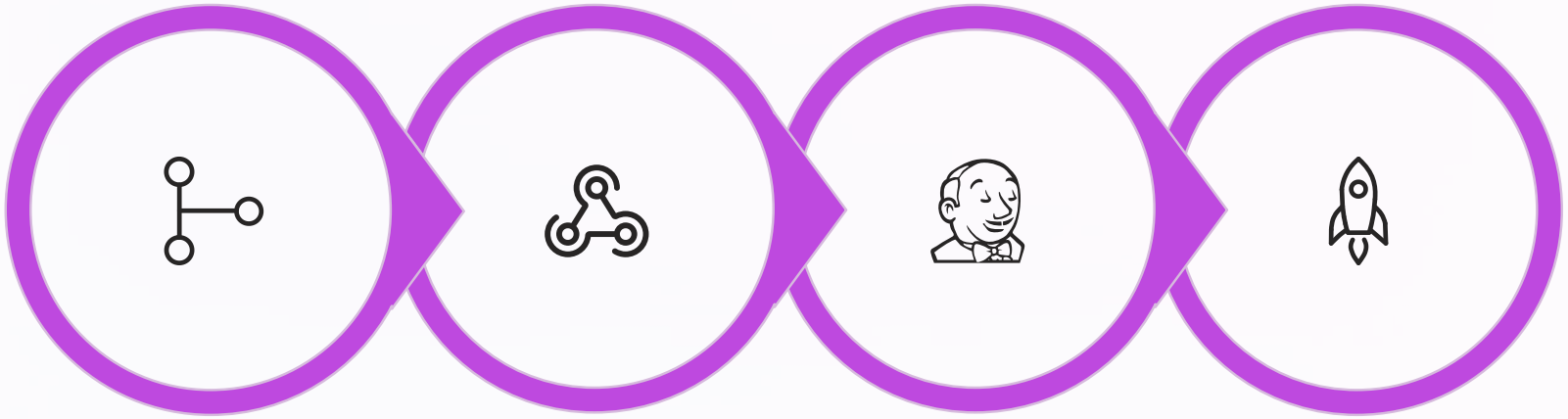
A **webhook** is an HTTP callback. When an event occurs on GitHub (like a push), GitHub sends a POST request to a URL you specify — in this case, your Jenkins server.

- Near real-time — no polling needed
- Triggers only on relevant events
- Payload contains commit details, branch, author

Jenkins must be publicly reachable (or via ngrok in local setups) to receive webhook calls.

BUILD TRIGGERS · CHAPTER 3

GitHub → Jenkins Trigger Flow



This automated loop means every code push is validated instantly — no manual intervention required.

BUILD TRIGGERS · CHAPTER 3

Configuring Build Triggers in Jenkins

In your job configuration under **Build Triggers**, enable the appropriate option:

1

GitHub hook trigger

Check **"GitHub hook trigger for GITScm polling"** — reacts to incoming webhooks automatically.

2

Poll SCM

Use cron syntax like `H/5 * * * *` to poll GitHub every 5 minutes. Fallback when webhooks aren't available.

3

Build periodically

Schedule nightly builds: `H 2 * * *` — useful for integration tests independent of commits.

GITHUB INTEGRATION · CHAPTER 4

Setting Up a GitHub Webhook

On GitHub

1. Go to your repo → **Settings → Webhooks**
2. Click **Add webhook**
3. Payload URL: `http://<jenkins-ip>:8080/github-webhook/`
4. Content type: `application/json`
5. Select **Just the push event** → Save

On Jenkins

1. Open job → **Configure**
2. Under **Build Triggers**, enable **GitHub hook trigger**
3. Save the job configuration
4. Push a commit to verify the trigger fires

A green checkmark on GitHub's webhook page means the handshake succeeded!

GITHUB INTEGRATION · CHAPTER 4

Triggering Builds on Push

Once configured, every `git push` to the tracked branch fires the Jenkins job automatically. The commit SHA, author, and branch are logged in the build metadata.

GITHUB INTEGRATION · CHAPTER 4

Common Webhook Issues

🔴 Jenkins Not Reachable

GitHub can't reach `localhost:8080`. Use **ngrok** to expose a tunnel: `ngrok http 8080`, then use the ngrok URL in the webhook.

🔴 Missing Trailing Slash

The webhook URL **must** end with `/github-webhook/` — the trailing slash is required by Jenkins.

🔴 Plugin Not Installed

Ensure the **GitHub plugin** is installed in Jenkins → Manage Jenkins → Plugins.

GITHUB INTEGRATION · CHAPTER 4

Expected Behavior After a Push

Push Happens

Developer runs `git push origin main`

Webhook Fires

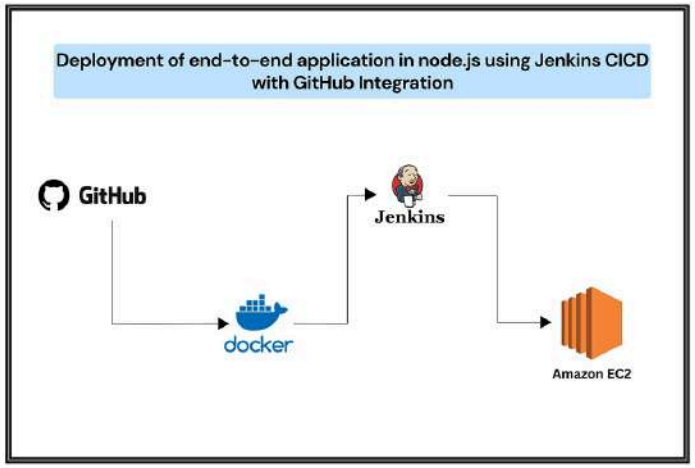
GitHub POSTs to Jenkins within seconds

Build Queued

Job appears in Jenkins build queue immediately

Result Reported

Build status shown on GitHub commit and Jenkins dashboard



PIPELINE AS CODE · CHAPTER 5

What Is a Jenkins Pipeline?

A Jenkins Pipeline is a suite of plugins that supports implementing and integrating continuous delivery pipelines into Jenkins — defined entirely in code via a **Jenkinsfile**.

PIPELINE AS CODE · CHAPTER 5

Why Pipelines Over Freestyle Jobs?

❌ Freestyle Limits

- Config stored in Jenkins XML, not in Git
- No native multi-stage support
- Hard to review, audit, or replicate
- Breaks with Jenkins server migration

✅ Pipeline Advantages

- Jenkinsfile lives in your repo
- Full multi-stage, parallel support
- Code-reviewed like application code
- Survives Jenkins restarts and migrations

PIPELINE AS CODE · CHAPTER 5

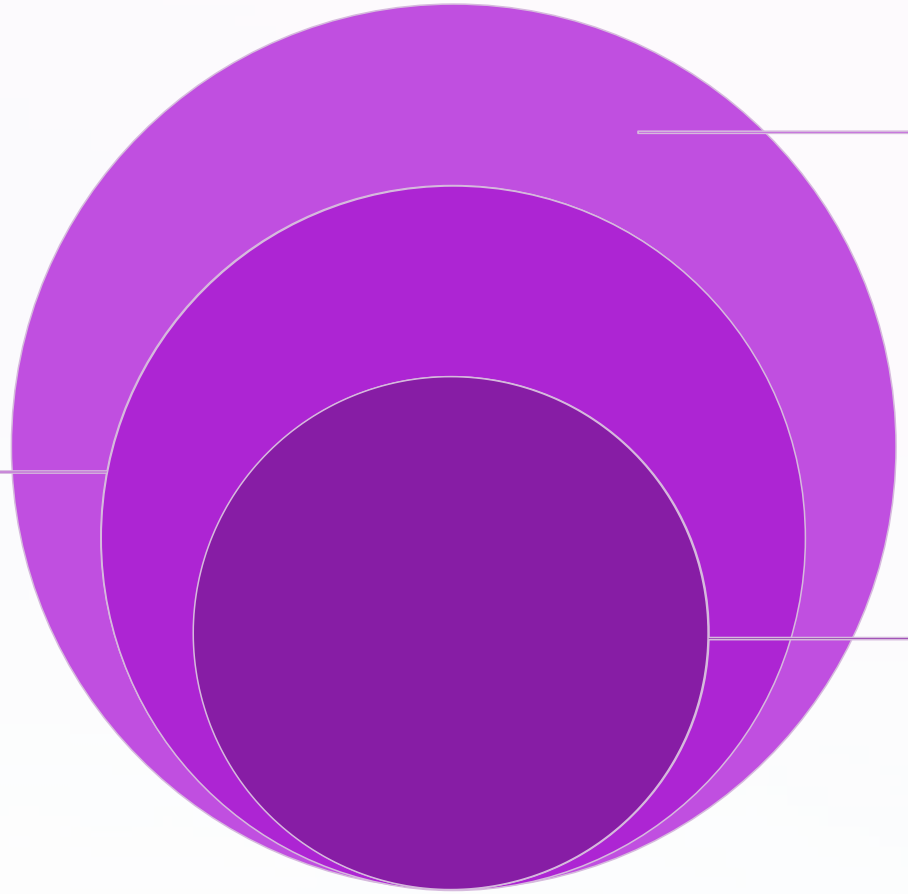
Declarative Pipeline Concept

Declarative pipelines use a structured, opinionated syntax that's easier to read and write than scripted pipelines.

```
pipeline {
  agent any        // Run on any available agent
  stages {
    stage('Build') {
      steps {
        echo 'Building the app...'
      }
    }
    stage('Test') {
      steps {
        echo 'Running tests...'
      }
    }
  }
}
```

Declarative is the **recommended** syntax for all new pipelines. Scripted pipelines offer more flexibility but require Groovy knowledge.

Pipeline Structure Overview



Steps

Shell commands and actions inside each stage

Stages

Contains Build, Test, Deploy stage blocks

Pipeline Block

Defines agent and top-level stages

Every Declarative Pipeline follows this nesting hierarchy. Understanding the structure prevents 90% of syntax errors.

JENKINSFILE DEEP DIVE · CHAPTER 6

The agent Directive

What It Does

Defines **where** the pipeline (or stage) runs — on which node, Docker container, or Kubernetes pod.

```
pipeline {
  // Option 1: any node
  agent any

  // Option 2: Docker container
  agent {
    docker {
      image 'node:18-alpine'
    }
  }

  stages { ... }
}
```

`agent any`
Run on any available Jenkins agent

`agent none`
No global agent; each stage defines its own

`agent { docker }`
Run inside a specified Docker image

JENKINSFILE DEEP DIVE · CHAPTER 6

Stages & Steps

Stages group related work into named phases visible in the pipeline visualization. **Steps** are the individual commands inside each stage.

```
stages {
 stage('Build') {
 steps {
 sh 'mvn clean package -DskipTests'
 echo "Build artifact: target/app.jar"
 }
 }
 stage('Test') {
 steps {
 sh 'mvn test'
 junit 'target/surefire-reports/*.xml'
 }
 }
 stage('Deploy') {
 steps {
 sh 'scp target/app.jar user@server:/deploy/'
 }
 }
}
```

JENKINSFILE DEEP DIVE · CHAPTER 6

Build, Test & Deploy Stages

🔨 Build

Compile source code, resolve dependencies, generate artifacts. Fails fast on compile errors before wasting test time.

🧪 Test

Run unit, integration, and linting checks. Publish test reports so failures are traceable to specific assertions.

🚀 Deploy

Ship the validated artifact to staging or production. Often gated by manual approval in production environments.

JENKINSFILE DEEP DIVE · CHAPTER 6

Complete Sample Jenkinsfile

```
pipeline {
 agent any
 environment {
 APP_NAME = 'my-web-app'
 VERSION = '1.0.0'
 }
 stages {
 stage('Checkout') {
 steps { checkout scm }
 }
 stage('Build') {
 steps { sh 'npm install && npm run build' }
 }
 stage('Test') {
 steps { sh 'npm test -- --coverage' }
 }
 stage('Docker Build') {
 steps {
 sh "docker build -t ${APP_NAME}:${VERSION} ."
 }
 }
 stage('Deploy') {
 steps { sh './scripts/deploy.sh' }
 }
 }
 post {
 success { echo '
```

JENKINSFILE DEEP DIVE · CHAPTER 6

The post Block

The post section defines actions that run **after** pipeline completion, regardless of outcome.

always

Runs on every build — clean up workspaces, archive logs

success

Notify Slack, tag Docker image, trigger downstream jobs

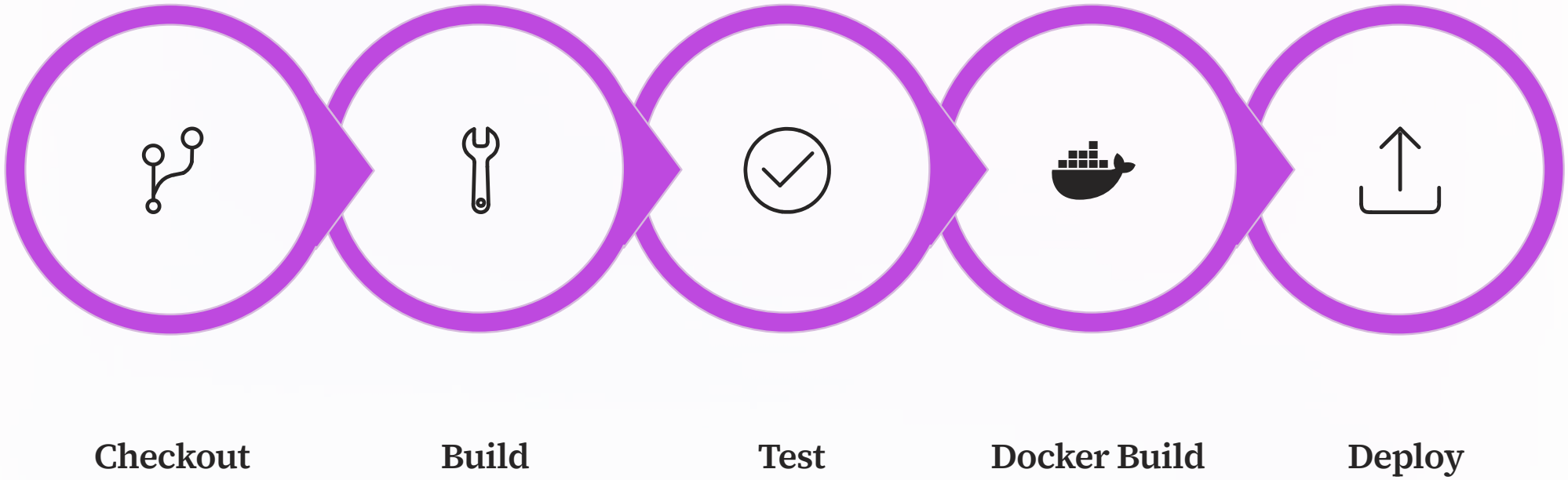
failure

Send alert email, open ticket, rollback deployment

unstable

Some tests failed — notify team without blocking pipeline

Pipeline Execution Visualization



Jenkins renders this stage view automatically in the Blue Ocean UI. Each box shows duration and pass/fail state at a glance.

DOCKER INTEGRATION · CHAPTER 7

Why Docker in CI/CD?

🌍 Consistency

Same container runs locally, in CI, and in production — eliminating "works on my machine" problems.

🔒 Isolation

Each build runs in a fresh container. No dependency bleed between jobs or builds.

📦 Portability

The Docker image IS the deployable artifact — ship the same image from dev through production.

DOCKER INTEGRATION · CHAPTER 7

Building a Docker Image in Your Pipeline

Add a **Docker Build** stage to your Jenkinsfile. Jenkins must have Docker installed on the agent and the user must have Docker socket access.

```
stage('Docker Build') {
 steps {
 script {
 def imageName = "myapp:${env.BUILD_NUMBER}"

 // Build the image using the repo's Dockerfile
 sh "docker build -t ${imageName} ."

 // Verify the image was created
 sh "docker images | grep myapp"

 echo "
```

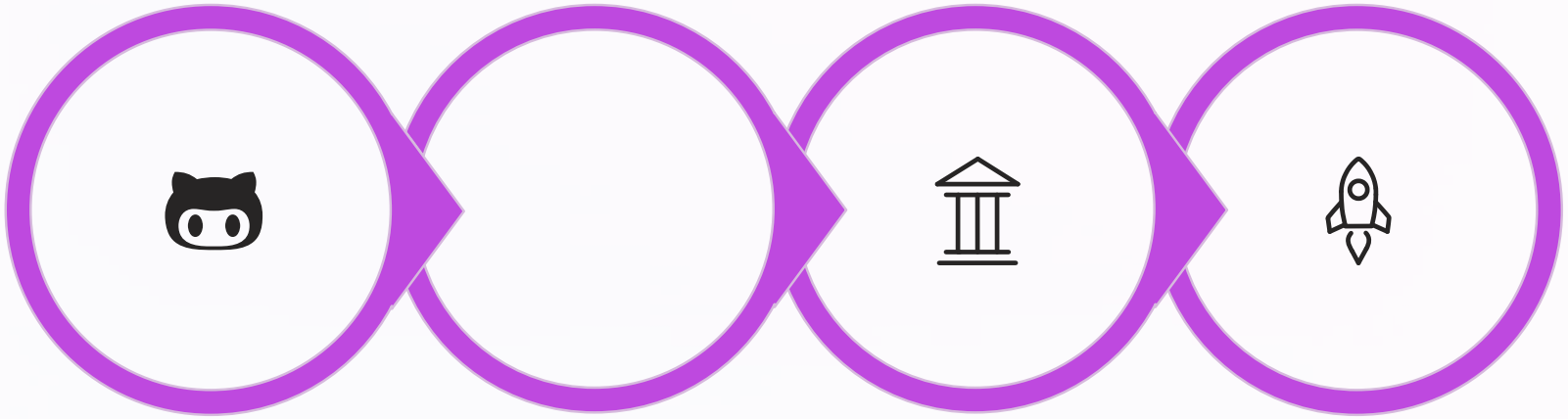
DOCKER INTEGRATION · CHAPTER 7

Running a Container from the Pipeline

```
stage('Docker Run & Smoke Test') {
 steps {
 script {
 def imageName = "myapp:${env.BUILD_NUMBER}"

 // Start container in detached mode
 sh "docker run -d -p 3000:3000 --name test-${BUILD_NUMBER} ${imageName}"

 // Wait for app to start
 sh 'sleep 5'

 // Basic smoke test
 sh 'curl -f http://localhost:3000/health || exit 1'

 // Cleanup
 sh "docker rm -f test-${BUILD_NUMBER}"
 }
 }
}
```

DOCKER INTEGRATION · CHAPTER 7

End-to-End Flow: GitHub → Jenkins → Docker



This is the core CI/CD loop. Every push is automatically built, containerized, and validated — ready to ship in minutes.

AI INTEGRATION · CHAPTER 8

AI Enters the CI/CD Pipeline

Modern DevOps engineers use **ChatGPT** and **GitHub Copilot** to accelerate Jenkinsfile authoring, diagnose failures, and write better pipeline code — faster than ever before.

AI INTEGRATION · CHAPTER 8

Generate a Jenkinsfile with ChatGPT

Example Prompt

> "Write a declarative Jenkins pipeline for a Node.js app that: checks out from GitHub, runs `npm install` and `npm test`, builds a Docker image tagged with the build number, and sends a Slack notification on failure."

ChatGPT will produce a production-ready Jenkinsfile in seconds — review, adapt, and commit.

✅ What AI Does Well

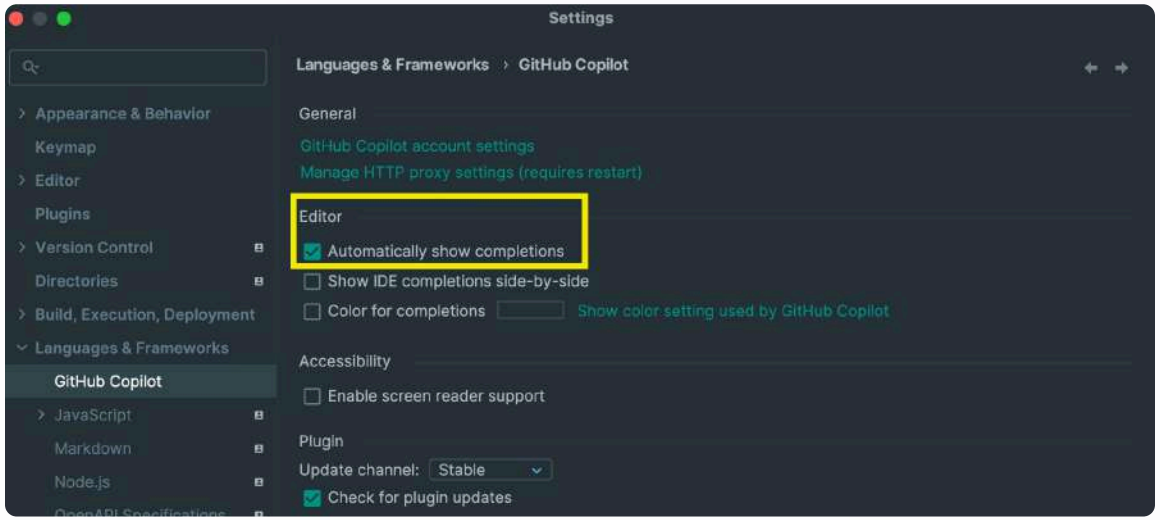
- Boilerplate pipeline structure
- Plugin syntax lookups
- Environment variable patterns
- Post-section notification code

⚠️ Always Verify

- Credentials handling
- Security-sensitive steps
- Plugin version compatibility

AI INTEGRATION · CHAPTER 8

GitHub Copilot for Pipelines



Copilot in Action

Open your `Jenkinsfile` in VS Code with Copilot enabled. As you type comments, Copilot suggests complete stage blocks.

- Type `// stage to build Docker image` → Copilot writes the stage
- Type `// add parallel test execution` → Copilot adds the parallel block
- Use **Tab** to accept, **Esc** to reject suggestions

> Copilot learns from your existing Jenkinsfile context — the more you write, the better its suggestions get.

AI INTEGRATION · CHAPTER 8

Debugging Pipeline Errors with AI

Copy the failing console output and paste it into ChatGPT with a simple structured prompt.

🔍 Diagnostic Prompt

"This Jenkins pipeline stage failed with the following error. Explain what caused it and provide a fix:"

Then paste the relevant log lines.

🛠️ Fix & Explain Prompt

"Here is my Jenkinsfile stage and the error it produces. Rewrite the stage to fix the issue and explain what was wrong."

🔒 Security Review Prompt

"Review this Jenkinsfile for security issues — credentials exposure, insecure shell commands, or missing input validation."

AI INTEGRATION · CHAPTER 8

AI Prompt Templates for DevOps

Goal	Prompt Template
Generate pipeline	"Write a Jenkins declarative pipeline for [language/framework] with build, test, and Docker stages."
Debug error	"This Jenkins error occurred: [paste error]. What does it mean and how do I fix it?"
Optimize stage	"Optimize this Jenkins stage for speed. Current version: [paste code]."
Add notifications	"Add Slack notifications to this Jenkinsfile on failure and success."
Explain syntax	"Explain what this Jenkinsfile block does in plain English: [paste block]."

FAILURE HANDLING · CHAPTER 9

Common Pipeline Errors

🔴 No such file or directory

Working directory mismatch. Use `sh 'ls -la'` to debug, or set `dir('subdir') { ... }` explicitly.

🔴 Permission denied

Jenkins user lacks Docker socket access. Fix: `sudo usermod -aG docker jenkins` then restart Jenkins.

🔴 Script not approved

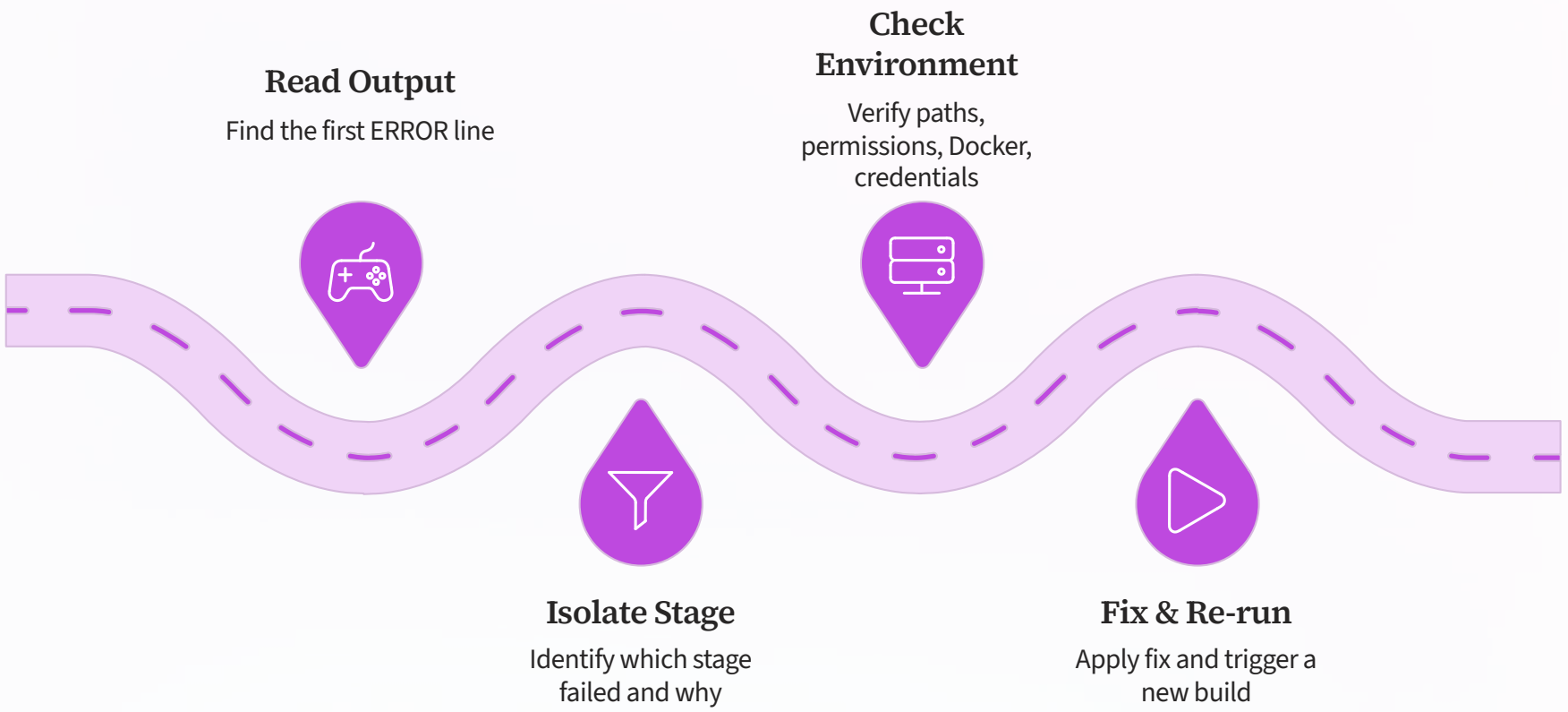
Groovy sandbox blocked a method. Go to **Manage Jenkins → In-process Script Approval** to approve.

🔴 Agent offline

No agents available matching the label. Check agent status under **Manage Jenkins → Nodes**.

FAILURE HANDLING · CHAPTER 9

Systematic Debugging Workflow



Resist the urge to randomly change things. Follow this systematic process — most pipeline failures are resolved in the first two steps.

FAILURE HANDLING · CHAPTER 9

AI-Assisted Troubleshooting: Live Example

The Error

```
+ docker build -t myapp:42 .
Got permission denied while trying
to connect to the Docker daemon
socket at unix:///var/run/docker.sock:
dial unix /var/run/docker.sock:
connect: permission denied
```

The AI-Suggested Fix

```
# Add jenkins user to docker group
sudo usermod -aG docker jenkins

# Restart Jenkins service
sudo systemctl restart jenkins

# Verify group membership
groups jenkins
# Should include: jenkins docker
```

This exact error stumps beginners for hours. AI identifies the root cause in seconds.

DAY 4 · KEY TAKEAWAYS

What You Mastered Today

Jenkins Jobs

Freestyle vs Pipeline, job anatomy, SCM integration

GitHub Webhooks

Push-triggered builds, end-to-end automation loop

Jenkinsfile

Declarative syntax, stages, steps, post blocks

Docker in CI/CD

Build, run, and test containers from your pipeline

AI Assistance

ChatGPT prompts, Copilot, AI-powered debugging

Day 5 Preview: Advanced Jenkins & Capstone

🔧 Advanced Jenkins

Shared libraries, multi-branch pipelines, parallel stages, and matrix builds

🤖 Deep AI Debugging

AI-driven log analysis, automated root cause detection, Copilot pair programming

🏆 Capstone Project

Build a complete CI/CD pipeline from scratch: GitHub → Jenkins → Docker → Deploy

Capstone Project Brief

Your Mission

Assemble everything from Days 1–4 into a single working pipeline for a real application.

Fork the starter repo

Clone the provided sample app from GitHub

Write the Jenkinsfile

Author a declarative pipeline with 4+ stages

Integrate Docker

Build and run a container as part of the pipeline

Use AI to debug

Intentionally break and fix the pipeline with AI help

See You on Day 5 🚀

Practice today's labs, review your Jenkinsfile, and come ready to build something real.

Great DevOps engineers ship code — they don't just talk about it.